

A Spiking Transfer Learning Method for Event Camera Based on Structural Decoupling

1st Peiming Kan
Complex Laboratory of
New Finance and Economics,
Southwestern University of
Finance and Economics
Chengdu, China
telkeep@163.com

Abstract—Event cameras are widely used in high-speed, low-exposure scenarios due to their high dynamic range and high temporal resolution. Spiking Neural Networks (SNNs), with their event-driven and energy-efficient characteristics, are particularly suitable for training on event data. However, compared with frame-based image datasets, the labeled event data is insufficient for training SNN models with comparable performance. Existing research typically addresses this issue by optimizing the architecture and training process of SNN models, rarely considering the physical properties of event data. Specifically, compared to detailed texture information, frame-based images and event data share higher structural similarity. Therefore, we propose a Structural Decoupling-Based Spiking Transfer Learning Method (SDSTL), which leverages frame-based structural knowledge to assist in training SNNs on event data. We achieved a notable improvement in classification accuracy on two neural datasets compared to the baseline.

Index Terms—Event camera, Edge detection, Transfer learning, Spiking neural network

I. INTRODUCTION

Event cameras [1] are a new type of camera that has emerged in recent years. Event cameras record the light change states of pixels with high dynamic range, low power consumption, and high temporal resolution advantages. It differs from standard cameras, which capture images with rich texture and high resolution at a fixed frame rate. Standard cameras have significant limitations in extreme exposure scenarios (e.g., low light, high-speed motion) in which event cameras are capable of capturing more information. Meanwhile, event cameras have been researched for various vision tasks such as feature detection and tracking, optical flow estimation, 3D reconstruction, and pose estimation [2]–[5]. The event streams output by event cameras are typically sparse and non-uniform. Their high temporal resolution also results in a larger volume of data, which makes traditional models and vision algorithms unsuitable for downstream tasks in event cameras.

Spiking Neural Networks, as the third generation of neural networks, simulate the spiking computation of biological neurons, which has a similar neuromorphic mechanism as event cameras. Using SNNs for event data classification has been

a popular approach. However, compared to the rich frame-based image dataset, event data is limited by the availability of labeled data. Researchers have attempted to optimize various modules in classification training to improve the efficiency of event data [6]–[9]. Some works have explored zero-shot learning to increase the utilization of limited event data [10], or directly used simulators [11] to convert frame-based data into event stream data for downstream tasks [12].

However, these works have not focused on the physical properties of event data itself. Specifically, compared to detailed texture information, frame-based images and event data share higher structural similarity. This provides a bridge of knowledge transfer from label-rich frame-based images to event data. Structural features primarily include the shape, outline, and relative position of objects within a scene. These features are crucial for understanding the main objects in a scene and their dynamic changes. These structural features are generally more stable and recognizable than detailed texture information. This suggests that we can attempt to transfer the structural knowledge learned from frame-based images to event data to improve event data classification.

Based on this inspiration, in this work, we propose a knowledge transfer method based on edge detection and design a training framework to guide event data classification, called SDSTL (Structural Decoupling-Based Spiking Transfer Learning Method). We use edge detection techniques to decouple frame-based images. The centered kernel alignment (CKA) and temporal CKA (TCKA) loss function [13] is introduced to align the decoupled edge image and the frame-based image distributions. This alignment helps obtain pre-trained encoder parameters, which are then used to guide the subsequent training. The approach achieves the goal of structural knowledge transfer. With the help of this training framework, the classification efficiency of SNNs on neuromorphic datasets is improved.

II. PRELIMINARY AND RELATED WORK

A. Leaky Integrate-and-Fire (LIF) neuron model

The basic unit of an SNN is the spiking neuron. Among these, the Leaky Integrate-and-Fire (LIF) [14] model is the most widely used.

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62376228.

Guisong Liu is the corresponding author.

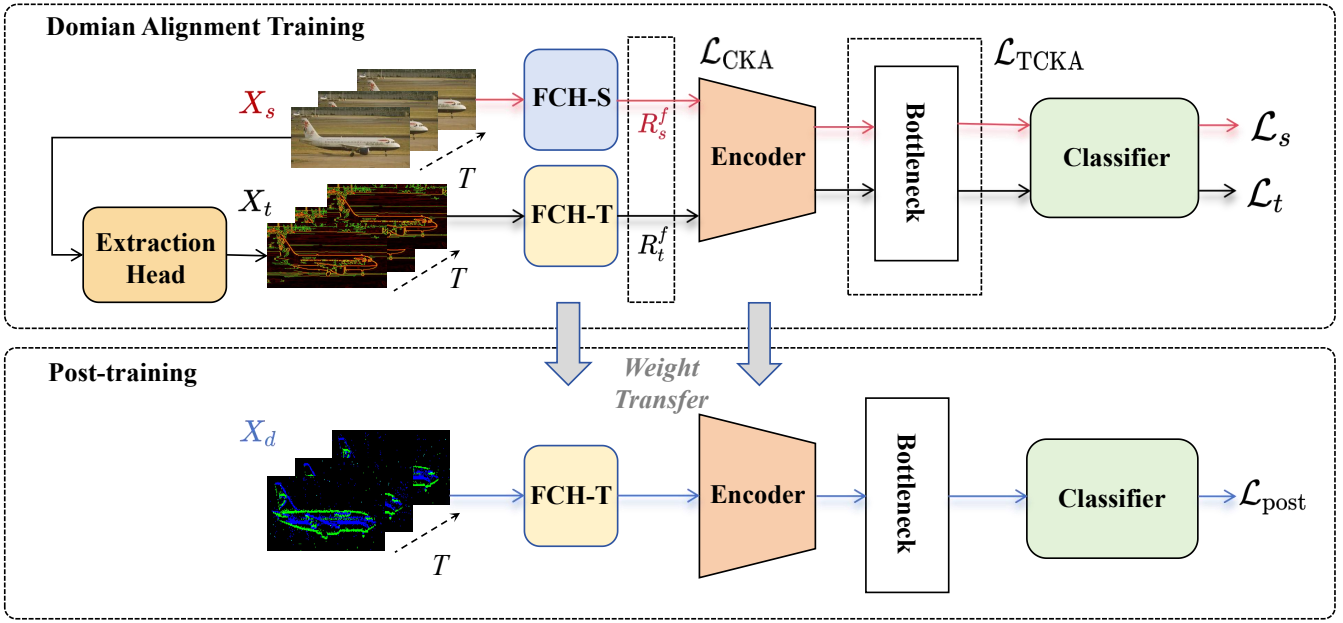


Fig. 1. Overall architecture diagram of SDSTL. During the domain alignment training stage, frame-based images and edge maps alternate through the model for structural deconstruction training. Parameter migration occurs partially, and the post-training model is consistent with the baseline model. FCH-S and FCH-T modules map the data of the two modalities to the high-order feature space of the same channel. R_s^f and R_t^f represent the average firing rate of the last layer of neurons in the FCH module at time step T . The bottleneck layer is a spiking linear layer with 512 input and 256 output neurons.

At each time step τ , the LIF neuron updates its membrane potential U_τ based on the membrane potential $U_{\tau-1}$ from the previous time step and the current input x_τ . The following equation represents this process:

$$U_\tau = U_{\tau-1} + \frac{1}{\alpha} (U_{\text{reset}} - U_{\tau-1} + Wx_\tau), \quad (1)$$

where W is the synaptic weight matrix, α is the time constant that controls the leakage rate of the membrane potential over time, and U_{reset} is the reset potential of the neuron.

When the membrane potential U_τ exceeds the firing threshold θ , the neuron will spike. The occurrence of the spike is determined by the Heaviside step function:

$$S_\tau = \text{sign}(U_\tau - \theta) = \begin{cases} 0, & U_\tau - \theta < 0, \\ 1, & U_\tau - \theta \geq 0. \end{cases} \quad (2)$$

where $S_\tau \in \{0, 1\}$ indicates whether a spike has occurred, with 1 for spiking and 0 for no spike.

The reset mechanism of the membrane potential is updated based on whether a spike is fired. If a spike is fired, the membrane potential is reset to U_{reset} . Otherwise, it remains at its current value. The following equation describes this process:

$$U'_\tau = (1 - S_\tau) \cdot U_\tau + S_\tau \cdot U_{\text{reset}}. \quad (3)$$

B. Event Data Classification by SNN Methods

Compared to frame-based image datasets, event data is relatively lacking in labeled samples, making it more challenging to train SNN models to achieve comparable performance. A

predominant approach in the literature is to enhance classification performance by directly optimizing the architecture and training mechanisms of SNNs. These efforts have led to notable improvements by focusing on various components of the SNN models.

One line of research focuses on refining the fundamental neuron model and the training algorithms [15]. For instance, to improve the information capacity of SNNs, Wu et al. proposed the Leaky Integrate and Analog Fire (LIAF) neuron model, which replaces conventional binary spikes with analog values for intercellular communication, thereby enriching the information flow [8]. To enable the training of deeper and more powerful SNNs, Zheng et al. introduced the Spatiotemporal Backpropagation (STBP) algorithm, which effectively mitigates the gradient vanishing and explosion problems inherent in deep network training [6]. Similarly, to enhance the generalization capabilities of deep SNNs, Deng et al. designed the Temporal Efficient Training (TET) method, which optimizes the gradient descent process to achieve faster and more robust convergence [9]. Other works have developed frameworks centered on streaming rollouts to better process spatiotemporal data streams and ensure the effective integration of temporal information during training [7]. Some researchers have explored altering the connection pathways within SNNs to create more efficient or specialized network structures [16].

Another idea is to optimize the learning mechanism. Lungu et al. proposed a multi-resolution Siamese network with hybrid training, combining similarity and classification tasks to enhance feature discrimination across diverse datasets [10]. Their architecture integrates multi-scale kernels and achieves

efficient inference on edge devices. Others have adopted hybrid fusion techniques, which combine SNNs with other neural network architectures to leverage the strengths of different modeling paradigms [17].

While these methods have significantly advanced the state-of-the-art in event data classification, their primary focus remains model-centric. That is, they achieve performance gains by improving the SNN model’s structure, neuron dynamics, or training strategy. In contrast, our work takes a different perspective based on domain adaptation [18]. We argue that substantial improvements can be gained by shifting the focus from the model to the data itself. Our approach seeks to explicitly exploit the unique spatiotemporal characteristics inherent in event data, aiming to develop a more effective classification framework that is fundamentally data-driven rather than purely model-driven.

III. METHOD

A. Overview

In this section, we first introduce the data flow and the overall training process, which is divided into two parts: domain alignment training and post-training. Then, we introduce how we extract the rich structural knowledge of frame-based images in III-C. Finally, we elaborate on the composition of the loss function in III-D and how it is calculated.

B. Two-stage training

To enhance the SNN model’s capacity to capture the structural knowledge of frame-based images, we constrained the spatiotemporal features of the two types of data to facilitate the classification training of event data.

We refer to the first stage of training as domain alignment training. As shown in the top part of Fig. 1, the purpose of domain alignment training is to train an encoder that focuses on structural information, thereby providing a good structural prior for subsequent event data training. To this end, we employ a transfer learning approach, where frame-based image X_s is treated as the source domain and the edge map X_t is the target domain. The method of obtaining the edge map is introduced in III-C.

There are differences in data form and distribution between the frame-based images and edge maps, which makes direct transfer learning difficult. Therefore, before the encoder, we introduce two feature conversion heads named FCH-S and FCH-T to map them to a unified high-level feature space. Specifically, the one above maps the X_s from 3 channels to 64 channels, and the following feature conversion header maps the X_t from 2 channels to 64 channels. Both feature conversion heads are a layer of an SNN convolutional layer.

As shown in Fig. 1, we use the central kernel alignment loss $\mathcal{L}_{CKA}$ to constrain the outputs of the two feature conversion heads to be similar in distribution. After that, the two features are passed through the encoder, bottleneck layer, and classifier in sequence, and the two branches share the same weights. We compute the temporal central kernel alignment $\mathcal{L}_{TCKA}$ after the bottleneck layer to guide the optimization direction

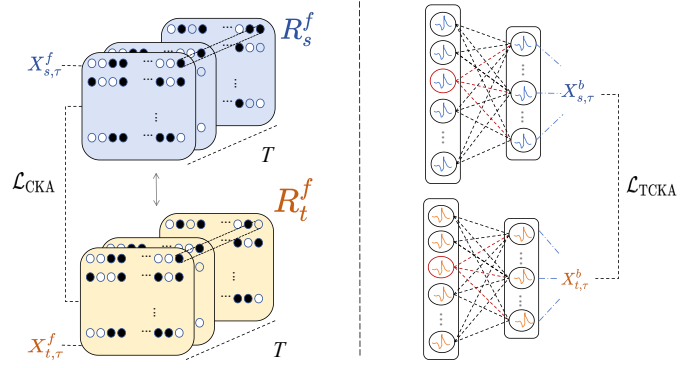


Fig. 2. The left side represents the output of the feature conversion head, and the calculation of the CKA loss function is based on the neuron firing rates; the right side represents the membrane potential of the bottleneck layer when the source domain and target domain features pass through, and the calculation of TCKA is based on membrane potentials.

of the SNN backbone network to provide a finer distribution alignment. After completing the domain alignment, we use the membrane potential of the last layer classifier to calculate the classification loss $\mathcal{L}_s$ and $\mathcal{L}_t$ of the source and target domains by TET [9], respectively.

Algorithm 1 Two-Stage Training

Input: frame-based dataset $\{\mathbf{I}, y\}$, event dataset $\{X_d, y_d\}$, loss weights λ_1, λ_2

Output: A final trained SNN model for event data

1: **Stage 1: Domain-Alignment training**

- 2: Initialize network modules: FCH-S, FCH-T, Encoder, Bottleneck, Classifier
- 3: **for** each training epoch **do**
- 4: Generate domain data $\mathcal{X}_t$ from frame-based image $\mathbf{I}$
- 5: Calculate total loss

$$\mathcal{L} \leftarrow \mathcal{L}_s + \mathcal{L}_t + \lambda_1 \mathcal{L}_{CKA} + \lambda_2 \mathcal{L}_{TCKA}$$

- 6: Update all network parameters by minimizing $\mathcal{L}$
- 7: **end for**

8: **Stage 2: Transfer and Post-training**

- 9: Load domain alignment training’s FCH-T and Encoder parameters into a new SNN model
 - 10: **for** each post-training epoch **do**
 - 11: **for** each sample (X_d, y_d) in the event dataset **do**
 - 12: Calculate the post-training loss $\mathcal{L}_{\text{post}}$
 - 13: Fine-tune SNN parameters by minimizing $\mathcal{L}_{\text{post}}$
 - 14: **end for**
 - 15: **end for**
 - 16: **return** the final trained SNN model
-

After the domain alignment training, we reuse the parameters of the FCH-T and encoder at the post-training stage. As shown in the bottom part of Fig. 1, in the post-training stage,

the representation of event data is similar to that of edge maps, processed through a 2-channel input. In event data, each event contains a timestamp and pixel coordinate information, and the accumulated events over a specific period form an event frame. The event data is represented as $X_d \in \mathbb{R}^{T \times H \times W \times 2}$, where T is the time step, representing the time sequence length of the event frame, $H \times W$ is the spatial dimension of the image, and 2 represents the two polarity channels of the event data. At the post-training stage, we only optimize the classification loss $\mathcal{L}_{\text{post}}$. The whole two-stage training process is shown in Algorithm 1.

C. Structural Information Extraction

In this section, we model the data processing and introduce an edge detection mechanism at the input stage. As shown in Fig. 3, the original image is a standard frame-based image, denoted as $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$, which is processed using the Sobel and Canny operators for edge detection, generating two grayscale images $\mathbf{E}_{\text{sobel}}, \mathbf{E}_{\text{canny}} \in \mathbb{R}^{H \times W}$. After stacking them along the channel dimension, a two-channel image with a frame-based structural representation is constructed as follows, $X_t = [\mathbf{E}_{\text{sobel}}, \mathbf{E}_{\text{canny}}] \in \mathbb{R}^{H \times W \times 2}$.

Our backbone network is based on the SNN model, which requires the introduction of time-sequenced image data expanded into a time sequence of length T . For the source domain data, the original frame-based image is duplicated into a time sequence of length T , described as $X_s \in \mathbb{R}^{T \times H \times W \times 3}$. The structural image is replicated along the time dimension, forming a four-dimensional tensor, $X_t \in \mathbb{R}^{T \times H \times W \times 2}$.

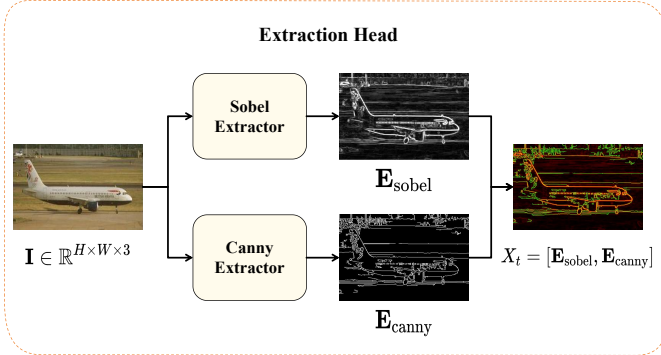


Fig. 3. This figure illustrates the process of generating a dual-channel structural image from a frame-based image after Sobel and Canny edge detection. To align with the event, we concatenate the two channels. For better visualization of the overlay effect, we map the combined image to a three-channel format.

D. Loss Functions

We use the CKA and TCKA methods to measure the similarity of features between the source domain input X_s and the target domain input X_t on different feature levels. Firstly, the calculation of $\mathcal{L}_{\text{CKA}}$ is based on the output of two feature conversion heads, as shown in the left half of Fig. 2. This loss function aims to keep the feature distributions of the source and target domains. Higher similarity corresponds

to lower loss. The features contain time step information, and we use the firing rate of neurons to calculate the loss:

$$\mathcal{L}_{\text{CKA}}(R_s^f, R_t^f) = 1 - \text{CKA}(R_s^f, R_t^f), \quad (4)$$

where $R_s^f = 1/T \sum_{\tau=1}^T X_{s,\tau}^f$ and $R_t^f = 1/T \sum_{\tau=1}^T X_{t,\tau}^f$. $X_{s,\tau}^f$ and $X_{t,\tau}^f$ represent the spiking output in the time step τ . CKA is defined as:

$$\text{CKA}(R_s^f, R_t^f) = \frac{\text{HSIC}(R_s^f, R_t^f)}{\sqrt{\text{HSIC}(R_s^f, R_s^f) \cdot \text{HSIC}(R_t^f, R_t^f)}}, \quad (5)$$

where HSIC is the Hilbert-Schmidt Independence Criterion [19].

After the features pass through the backbone network, a fine-grained constraint is provided, taking into account both the temporal and spatial dimensions to guide the adjustment of model parameters. As shown in the right half of Fig. 2, we use the $\mathcal{L}_{\text{TCKA}}$ to realize the work. It is defined as:

$$\mathcal{L}_{\text{TCKA}}(U_s^b, U_t^b) = 1 - \text{TCKA}(U_s^b, U_t^b), \quad (6)$$

where U_s^b and U_t^b are the average membrane potential of the bottleneck layer neurons. TCKA is an extension of CKA, considering the impact of the time dimension, and measures the similarity more elaborately:

$$\text{TCKA}(U_s^b, U_t^b) = \frac{1}{T} \sum_{\tau=1}^T \text{CKA}(U_{s,\tau}^b, U_{t,\tau}^b), \quad (7)$$

where $U_{s,\tau}^b$ and $U_{t,\tau}^b$ represent the membrane potential of the last layer of the bottleneck layer at the time step τ .

The last layer of the model is the classifier, which is an SNN linear layer. The classification loss is calculated using the membrane potential instead of spike output, which has better performance [9]. The calculation method is as follows:

$$\mathcal{L}_{\text{clf}} = \frac{1}{T} \sum_{\tau=1}^T \text{CE}(U_{\tau}^c, y), \quad (8)$$

where U_{τ}^c represents the membrane potential of the classifier neuron over the time step, and y is the target label of each domain. CE represents the cross-entropy loss. The classification loss of the source domain, target domain, and post-training is all calculated in this way. They are denoted by $\mathcal{L}_s$, $\mathcal{L}_t$ and $\mathcal{L}_{\text{post}}$, respectively.

Finally, the Domain Alignment Training objective function $\mathcal{L}$ is:

$$\mathcal{L} = \mathcal{L}_s + \mathcal{L}_t + \lambda_1 \mathcal{L}_{\text{CKA}}(R_s^f, R_t^f) + \lambda_2 \mathcal{L}_{\text{TCKA}}(U_s^b, U_t^b). \quad (9)$$

where λ_1 and λ_2 are the weighting coefficients for CKA and TCKA losses.

The post-training uses the domain alignment training's parameters and event data for classification tasks and optimizes the classification loss defined in Eq. (8).

IV. EXPERIMENTS

A. Datasets

To evaluate the effectiveness of our method, we conduct experiments on two image datasets, MNIST and CIFAR-10, and two event datasets, N-MNIST and DVS-CIFAR10. The **MNIST** dataset contains 70,000 28×28 grayscale images of handwritten digits, comprising 60,000 for training and 10,000 for testing, divided into 10 classes. **N-MNIST** is the neuromorphic version of MNIST, captured with an ATIS event camera at a resolution of 128×128 , which we also resized to 32×32 in this work. The **CIFAR-10** dataset consists of 60,000 32×32 color images across 10 classes, with 5,000 training images and 1,000 test images per class. The **DVS-CIFAR10** dataset is an event-based version of CIFAR-10, where original images are converted into event streams using an event sensor. Its original resolution is 128×128 , and we downsampled it to 32×32 for our experiments. We perform the Domain Alignment Training stage on image datasets and then conduct the post-training stage on event datasets, aiming to evaluate the performance of our method on event datasets.

B. Implementation Details

For our experiments, we train the SNN exclusively on event data using **VGGSNN** as the backbone network. The bottleneck layer is a spiking linear layer with 512 input and 256 output neurons, followed by a classification layer (also spiking linear) that reduces the dimension from 256 to 10 output neurons. The firing threshold θ was set to 1.0, and the membrane constant α was 0.5. The number of time steps T was set to 10. We employed the Adam optimizer with a default learning rate of 0.001, a batch size of 32, and a weight decay of 5×10^{-4} . The number of domain alignment training and post-training epochs both are 120. The loss coefficients λ_1 and λ_2 of the encoding alignment loss and feature alignment loss are both set to 0.1.

TABLE I
EXPERIMENT RESULTS FOR DIFFERENT METHODS AND MODELS ON N-MNIST

Method	Model	Accuracy (%)
SNN-IF [20]	Spiking CNN	99.31
DTS [21]	Spiking CNN	99.35
SLAYER [22]	Spiking CNN	99.20
BSNN [23]	Spiking CNN	99.50
LIAF [8]	LIAF-Net	99.13
TET ¹ [9]	VGGSNN	99.58
SDSTL (Our model)	VGGSNN	99.59

¹ Implemented by ourselves.

To ensure reproducibility, the random seed is fixed to 1000. All models are trained on an NVIDIA GeForce RTX 2080Ti GPU.

C. Experimental Results

We first analyze our experiments on the N-MNIST dataset. It is important to note that since MNIST images are single-

TABLE II
EXPERIMENT RESULTS FOR DIFFERENT METHODS AND MODELS ON DVS-CIFAR10

Method	Model	Accuracy (%)
STBP-ttBN [6]	ResNet-19	67.80
Streaming Rollout [7]	DenseNet	66.80
Conv3D [8]	LIAF-Net	71.71
LIAF [8]	LIAF-Net	70.43
TET ¹ [9]	VGGSNN	78.15
SDSTL (Our model)	VGGSNN	84.30

¹ Implemented by ourselves.

channel, we expanded them to three channels. Table I presents the performance of our SDSTL framework on the N-MNIST dataset. The results indicate that our two-stage framework improves the classification accuracy on N-MNIST, although this improvement is comparable to the original TET method. We attribute this to the relatively simple structure of the N-MNIST dataset, which may not have fully showcased the advantages of our SDSTL framework. Consequently, to better validate our framework's effectiveness, we investigate more complex datasets, CIFAR-10 and DVS-CIFAR10.

As shown in Table II, the methods focused on model optimization, such as STBP-ttBN [6] and LIAF [8], exhibited relatively poor classification performance on the event dataset. In contrast, by leveraging a deeper understanding of the data's intrinsic properties and introducing our two-stage framework, the SDSTL enabled the model to identify structural knowledge in event data better. This encourages the model to focus on the most salient information for the classification task, leading to a significant improvement in performance. These results demonstrate that by addressing the data's inherent characteristics with an effective two-stage strategy, our method can achieve superior classification performance on complex event datasets.

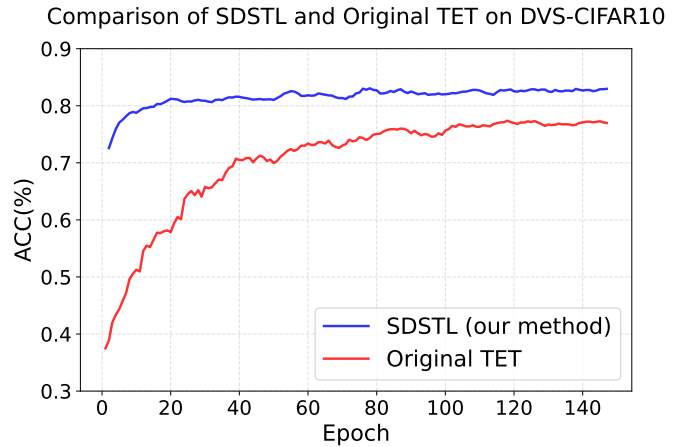


Fig. 4. Comparison with baseline model

We compare the convergence curves of our two-stage

method and the baseline TET method, as shown in Fig. 4. The results demonstrate that the SDSTL framework exhibits a significant advantage over the original TET on the DVS-CIFAR10 dataset. In terms of accuracy, SDSTL's performance rapidly increases and stabilizes at a high level, whereas the original TET remains at a comparatively lower level. These results confirm that SDSTL effectively leverages structural priors through transfer learning, enhancing both model performance and convergence efficiency.

V. CONCLUSION

In this paper, we propose the transfer learning framework SDSTL based on structural decoupling, which improves the classification performance of the SNN model on event data. This is achieved by extracting structural information from frame-based images and transferring this knowledge to the event domain. The SDSTL incorporates a structural alignment mechanism between edge maps and frame-based images, effectively identifies key information in event data, and accelerates the model's convergence process. Experimental results demonstrate that SDSTL outperforms existing methods on N-MNIST and DVS-CIFAR10 datasets, exhibiting superior efficiency and accuracy, particularly under complex datasets. For future work, we will explore more efficient structural extraction methods to enhance the transfer efficiency further. We also intend to extend this framework to other event-based vision tasks, such as object detection and tracking.

REFERENCES

- [1] G. Gallego, T. Delbrück, G. Orchard, C. Bartolozzi, B. Tabá, A. Censi, S. Leutenegger, A. J. Davison, J. Conradt, K. Daniilidis *et al.*, "Event-based vision: A survey," *IEEE transactions on pattern analysis and machine intelligence*, vol. 44, no. 1, pp. 154–180, 2020.
- [2] A. K. Kosta, M. P. E. Apolinario, and K. Roy, "Live demonstration: Ann vs snn vs hybrid architectures for event-based real-time gesture recognition and optical flow estimation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 4148–4149.
- [3] X. Qiu, M. Yao, J. Zhang, Y. Chou, N. Qiao, S. Zhou, B. Xu, and G. Li, "Efficient 3d recognition with event-driven spike sparse convolution," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 19, 2025, pp. 20 086–20 094.
- [4] A. Aydin, M. Gehrig, D. Gehrig, and D. Scaramuzza, "A hybrid ann-snn architecture for low-power and low-latency visual perception," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 5701–5711.
- [5] R. Wang, W. Jo, D. Zhao, W. Wang, A. Gupta, B. Yang, G. Chen, and B.-C. Min, "Husformer: A multimodal transformer for multimodal human state recognition," *IEEE Transactions on Cognitive and Developmental Systems*, vol. 16, no. 4, pp. 1374–1390, 2024.
- [6] H. Zheng, Y. Wu, L. Deng, Y. Hu, and G. Li, "Going deeper with directly-trained larger spiking neural networks," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 35, no. 12, 2021, pp. 11 062–11 070.
- [7] A. Kugele, T. Pfeil, M. Pfeiffer, and E. Chicca, "Efficient processing of spatio-temporal data streams with spiking neural networks," *Frontiers in neuroscience*, vol. 14, p. 512192, 2020.
- [8] Z. Wu, H. Zhang, Y. Lin, G. Li, M. Wang, and Y. Tang, "Liaf-net: Leaky integrate and analog fire network for lightweight and efficient spatiotemporal information processing," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 11, pp. 6249–6262, 2021.
- [9] S. Deng, Y. Li, S. Zhang, and S. Gu, "Temporal efficient training of spiking neural network via gradient re-weighting," in *International Conference on Learning Representations*.
- [10] I. A. Lungu, Y. Hu, and S.-C. Liu, "Multi-resolution siamese networks for one-shot learning," in *2020 2nd IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. IEEE, 2020, pp. 183–187.
- [11] Y. Hu, S.-C. Liu, and T. Delbruck, "v2e: From video frames to realistic dvs events," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 1312–1321.
- [12] K. Wang, S. Ma, J. Chen, F. Ren, and J. Lu, "Approaches, challenges, and applications for deep visual odometry: Toward complicated and emerging areas," *IEEE Transactions on Cognitive and Developmental Systems*, vol. 14, no. 1, pp. 35–49, 2020.
- [13] Q. Zhan, G. Liu, X. Xie, R. Tao, M. Zhang, and H. Tang, "Spiking transfer learning from rgb image to neuromorphic event stream," *IEEE Transactions on Image Processing*, vol. 33, pp. 4274–4287, 2024.
- [14] C. Teeter, R. Iyer, V. Menon, N. Gouwens, D. Feng, J. Berg, A. Szafer, N. Cain, H. Zeng, M. Hawrylycz *et al.*, "Generalized leaky integrate-and-fire models classify multiple neuron types," *Nature communications*, vol. 9, no. 1, p. 709, 2018.
- [15] Z. Han, Y. Shang, Z. Shao, J. Liu, G. Guo, T. Liu, H. Ding, and Q. Hu, "Spatial-temporal feature network for speech-based depression recognition," *IEEE Transactions on Cognitive and Developmental Systems*, vol. 16, no. 1, pp. 308–318, 2023.
- [16] X. Cheng, Y. Hao, J. Xu, and B. Xu, "Lisnn: Improving spiking neural networks with lateral interactions for robust object recognition," in *IJCAI*. Yokohama, 2020, pp. 1519–1525.
- [17] L. Deng, Y. Wu, X. Hu, L. Liang, Y. Ding, G. Li, G. Zhao, P. Li, and Y. Xie, "Rethinking the performance comparison between snns and anns," *Neural networks*, vol. 121, pp. 294–307, 2020.
- [18] D. Zhang, Z. Du, J. Li, L. Zhu, and H. T. Shen, "Domain-adaptive energy-based models for generalizable face anti-spoofing," *IEEE Transactions on Multimedia*, 2024.
- [19] A. Gretton, O. Bousquet, A. Smola, and B. Schölkopf, "Measuring statistical dependence with hilbert-schmidt norms," in *International conference on algorithmic learning theory*. Springer, 2005, pp. 63–77.
- [20] J. Wu, Y. Chua, M. Zhang, G. Li, H. Li, and K. C. Tan, "A tandem learning rule for effective training and rapid inference of deep spiking neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 1, pp. 446–460, 2021.
- [21] Y. Wu, L. Deng, G. Li, J. Zhu, Y. Xie, and L. Shi, "Direct training for spiking neural networks: Faster, larger, better," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 33, no. 01, 2019, pp. 1311–1318.
- [22] S. B. Shrestha and G. Orchard, "Slayer: Spike layer error reassignment in time," *Advances in neural information processing systems*, vol. 31, 2018.
- [23] G. Qiao, N. Ning, Y. Zuo, S. Hu, Q. Yu, and Y. Liu, "Direct training of hardware-friendly weight binarized spiking neural network with surrogate gradient learning towards spatio-temporal event-based dynamic data recognition," *Neurocomputing*, vol. 457, pp. 203–213, 2021.